

Big Data

INF 4040 TECHNOLOGIES FOR WEB-BASED INFORMATION SYSTEMS

AUBG COS DEPARTMENT

What is Big Data?

- The term big data refers to collections of **data sets** so **large** and **complex** that it is difficult to process this data using conventional relational database management systems (RDBMS) or data processing applications
- Instead, a new computing paradigm (massively **parallel software** running on tens, hundreds, or even thousands of servers) had to be developed

Using a Traditional Database – RDBMS/SQL

- Most RDBMS are designed for efficient **transaction processing** using **SQL** commands
 - Adding, updating, searching for, and retrieving relatively small amount of information
 - Data is acquired in a transactional fashion
- So, what is the problem with “Big Data”? – the three **Vs**
 - Large **volume** of data – at least tens of terabytes
 - Requiring high **velocity** – new data generated continuously, so must be analyzed quickly
 - Stored in a **variety** of formats – structured and unstructured

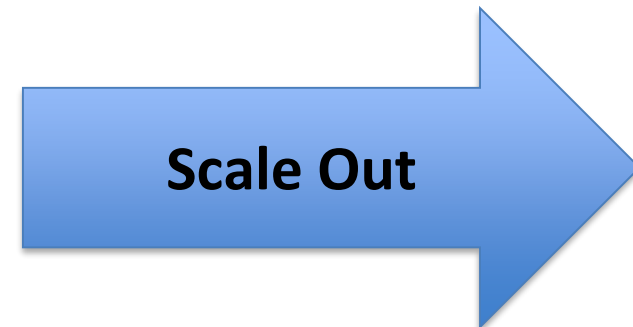
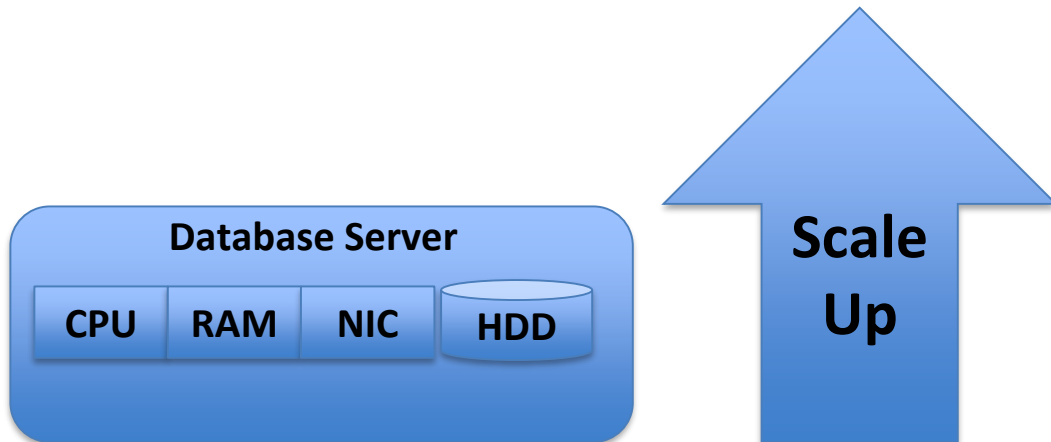
Big Data Applications

- In a big data application:
 - The number of hard-disk drives required would be huge for a single server
 - Number of processors required to process the data would be huge too
 - The format of much big data is not compatible with relational-type database records and schema
- So SQL and the relational model is not good for big data

What Can We Do To Accommodate Big Data?

- **Scale-up** the resources (**vertical** scaling)
 - Add Faster CPUs, Bigger Storage
 - **Not an option!**

- **Scale-out** resources (**horizontal** scaling)
 - Add more servers, and divide the task into smaller pieces and distribute among the servers
 - **Correct approach!**



Big Data Industry Trends

1. **Unstructured** data – data is moving from structured to unstructured format, raising the costs of analysis
2. **Open source** – proprietary database standards are giving way to new, open-source big data technology platforms such as **Hadoop**
3. **Cloud** – many corporations are opting to use cloud services to access big data analytical tools instead of building expensive data warehouses themselves
4. **M2M** – in future (IoT), a growing proportion of big data will be generated from **machine-to-machine** (M2M) using sensors

Internet of Things (IoT) and the Cloud

- It is projected that there will be **30 billion devices** on the Internet by 2030
- Most will be **small sensors** that send streams of information into the cloud where it will be processed and integrated with other streams and turned into knowledge that will help our lives in a multitude of small and big ways
- The cloud will become increasingly important as a controller of, and resource provider for, the **Internet of Things – the use of computers to enrich every aspect of everyday life**

Big Data Pioneers

- The **first businesses** that had to deal with big data were the leading Internet companies such as Amazon, Google, and Yahoo
- Google, for example, runs **web crawlers** which gather **unstructured data** – like web pages – and then process them within milliseconds to produce search **rankings**
- Worse still, they have to deal with millions of simultaneous users all submitting different search queries at once
- So Cloud developers designed **entirely new data store platforms** to deal with this type of unstructured query at lightning speed

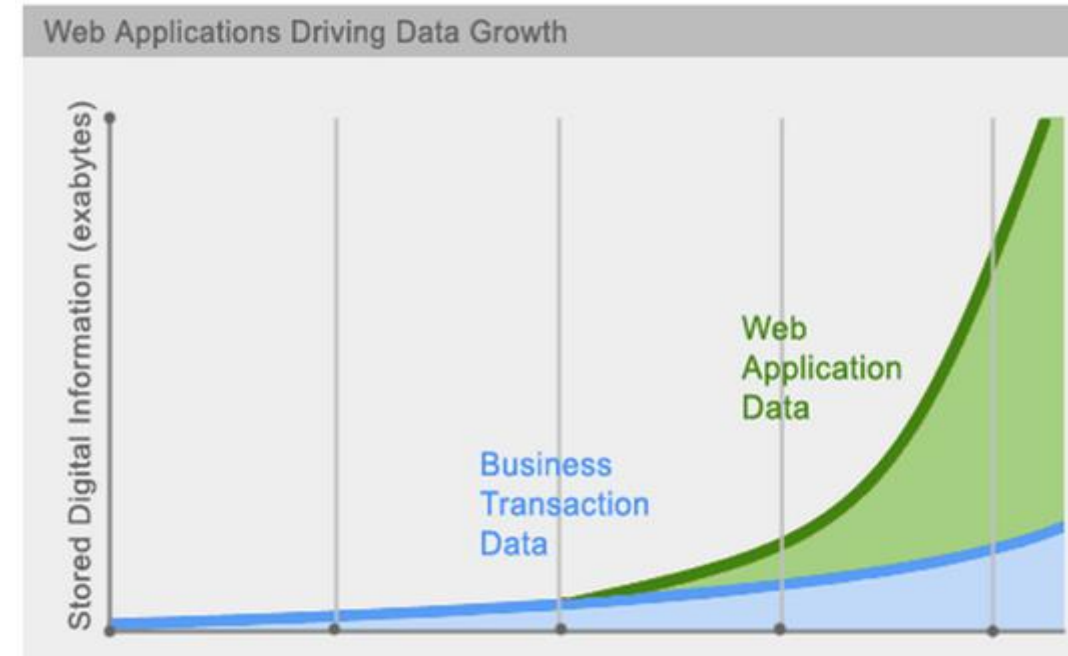
Big Data Pioneers (continued)

- The umbrella descriptive word “**NoSQL**” was coined to describe a number of different big data storage technologies that were developed for the Cloud
- Originally, this meant **No SQL**. But later its interpretation was softened to mean **Not Only SQL**



Data Usage Trends

- Today, data is becoming easier to access and capture through third parties such as Facebook, Google, etc
- Personal user information, social graphs, geo-location, user-generated content and machine logs are just a few examples of data **increasing exponentially**
- To analyze the data from the above services properly requires the processing of **huge amounts** of data



- **SQL is not good/designed for big data**

What is NoSQL?

- **NoSQL** technologies have evolved to handle this huge data properly
- NoSQL is the term used to designate **data store management systems** that **differ** from **traditional relational database** management systems
- These data stores **do not require** fixed-table schemas, usually **avoid join** operations, and typically **scale** horizontally

Why Not Use Relational DBs?

- An **RDBMS does not scale well** – adding more processors, disks, etc. to deal with tremendous amounts of data does not work
 - RDBMS not designed for a distributed computing environment
 - Complex SQL operations, e.g. JOINS are difficult to perform over multiple processors
- **Scaling up will not work** – single machine's resources have a limit that usually cannot accommodate the big data
- Researchers began looking at **multi-computer** data store solutions – aka **horizontal scaling** and distributed approach

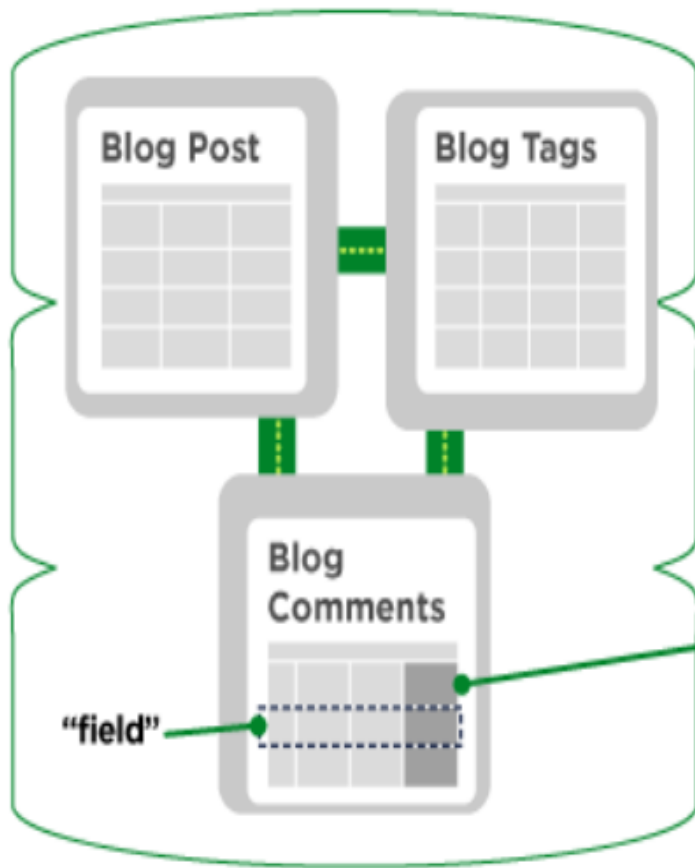
Horizontal Scaling Problem

- **Horizontal scaling** – adding more servers to the problem, and share the data set between the servers
 - **Each server** works on a **different portion** of the data
- But servers, hard disk drives, and other equipment can **break down**
 - Different portions of the data store may become inconsistent
- To solve that, **replicated servers** and data stores to deal with inconsistencies were introduced

NoSQL Features

- No declarative query language
- No predefined schema
- Format variants
 - Key-Value
 - Document-Store
 - Column Store
 - Graph
- Prioritizes high performance, high availability, and scalability

Relational Vs. Non-Relational Databases



A non-relational database does not incorporate the table model. Instead, data can be stored in a single document file.

A relational database table organizes structured data fields into defined columns.



NoSQL Distinguishing Characteristics

- **Large** data volumes
- Scalable **replication** and **distribution**
 - Potentially **thousands** of machines, distributed **around the world**
- Queries need to return answers **quickly**
- Mostly reads, **few updates**
- **Problem** – distributed data stores are not always consistent:
 - **Eventually consistent** – used in high availability systems, where approximate answers are OK

BASE

- Acronym contrived to be the opposite of ACID (atomicity, consistency, isolation, durability):
 - Basically Available: concurrent accessibility by users at all times. Even while the data is being modified
 - Soft state: data can have transient or temporary states while being updated so it is possible that different queries for a record see different values
 - Eventually Consistent: a record will achieve consistency when all the concurrent updates have been completed

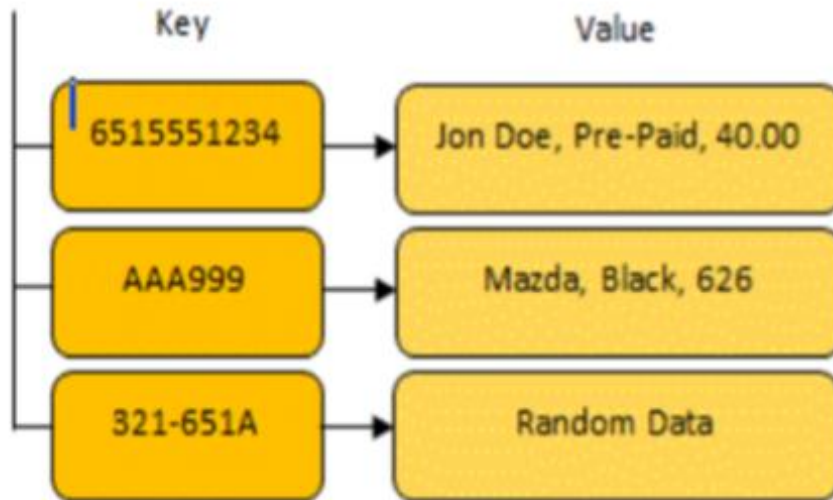
BASE

- Characteristics:
 - Weak consistency – stale data OK
 - Availability first
 - Best effort
 - Approximate answers OK
 - Simpler and faster

Different Types of NoSQL Systems

- **Distributed Key-Value Systems** – Lookup a single value using a given key – e.g. Amazon's DynamoDB
- **Document-based Systems** – Access data by key, or by search of “document” data – e.g. MongoDB
- **Column-based Systems** – e.g. Facebook's Cassandra
- **Graph-based Systems** – Use a graph structure - e.g. Google's Pregel, Neo4j
- **Use different types for different types of applications**

Key-Value Store



- Key - value pair;
- Each record is uniquely identified by the key;
- The data is stored in the value.
- Fast retrieval by key, no other way to get the data or to make any complicated queries;

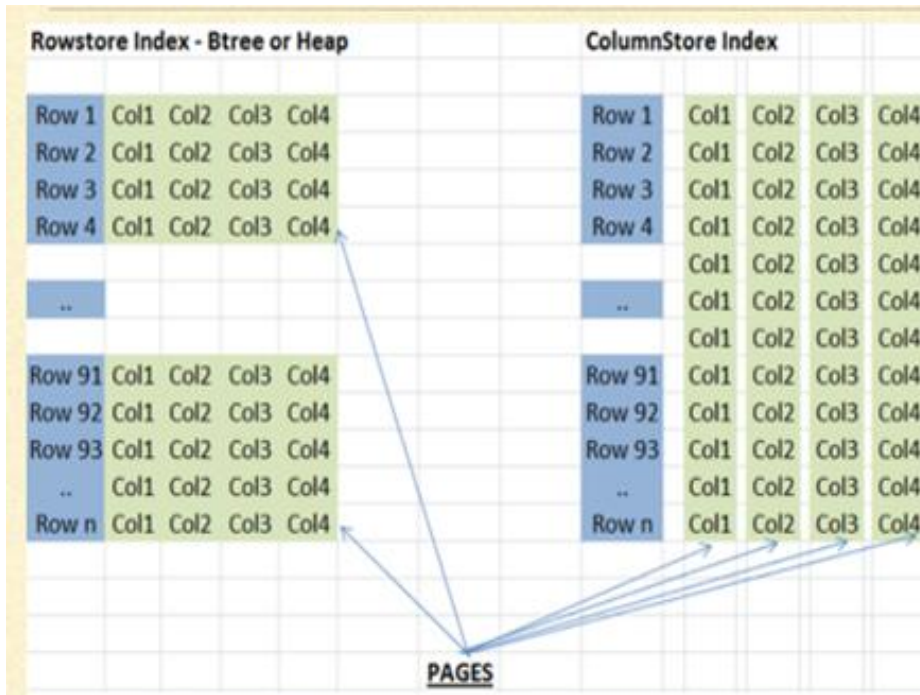
ORACLE®
NOSQL DATABASE



Key-Value Stores Features

- A Key-Value NoSQL data store keeps data as a set of Keys and Values, also known as an associative array, organized into rows
- Data model – (key, value) pairs
 - Key = string/integer, unique for the entire data
 - Value = Can be anything (very complex object)
- Operations
 - get(key), put(key, value)
 - Operations on value, such as search, not supported

Column Stores



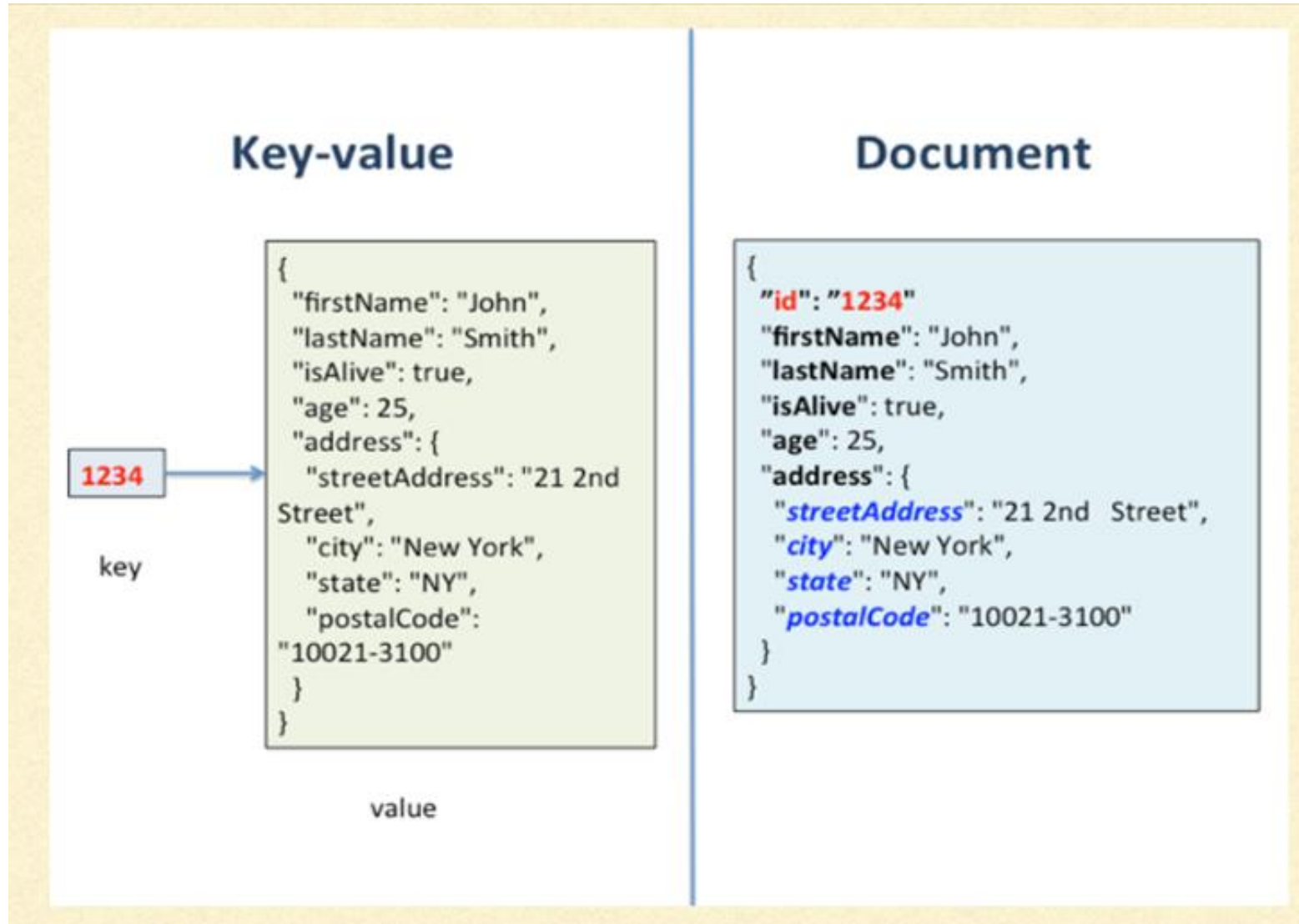
- They store each column separately, with attribute values belonging to the same column;
- Columns are in groups and those groups should be predefined;
- Indexing on the data;
- Searching by specific attribute is extremely fast.



Column Store Characteristics

- A **column store** uses tables, rows, and columns, like a RDBMS, but with values stored on the hard disks in **columns** rather than rows
- By storing data in columns rather than rows, the database performs less input/output and gets better compression ratios than conventional database systems that store data in rows. This is because similar data items are side-by-side, since they come from the same column

Document Store

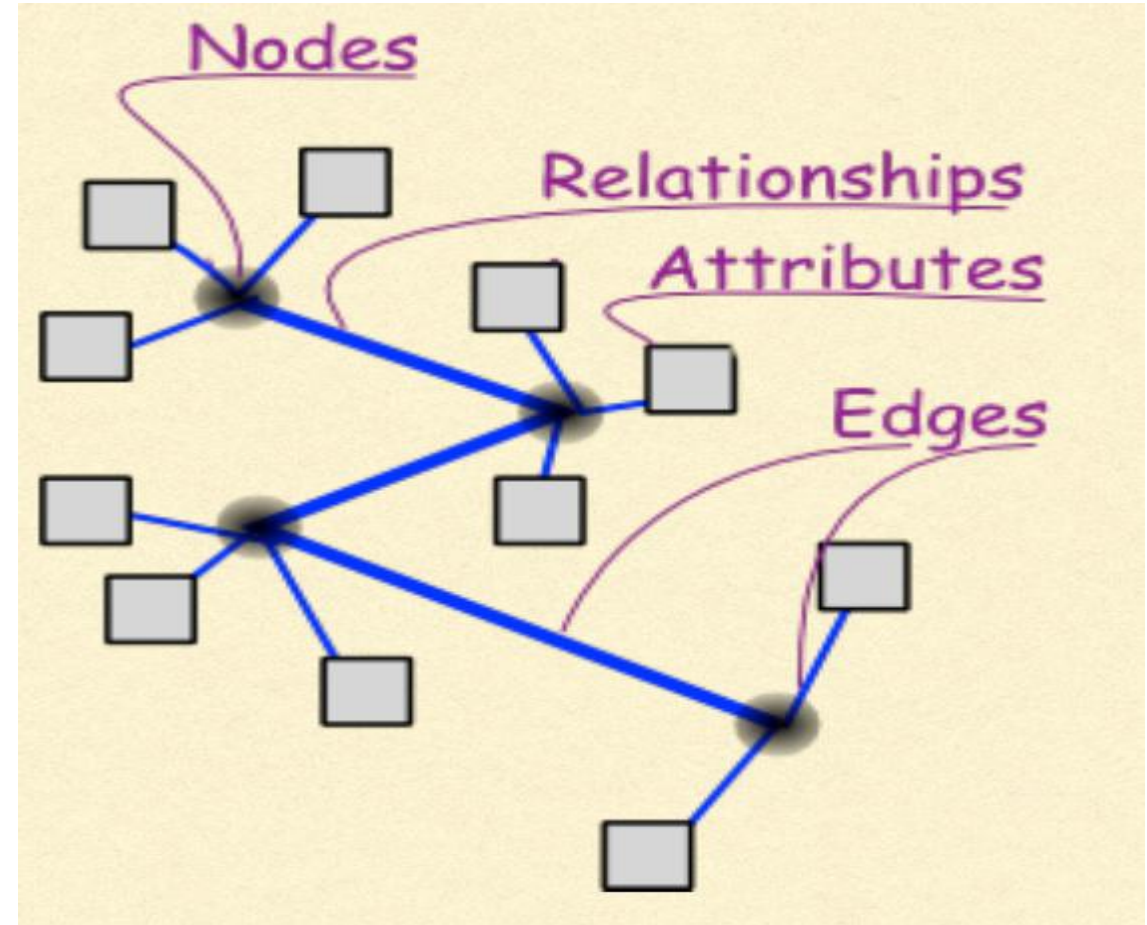


Document Stores Features

- In **Key-Value** stores, the value is often a very complex object
 - E.g. Key = '2019/7/1', Value = *[all flights that date]*
 - **No search** operations are allowed on the Value
- Data model for document store: **(key, document)** pairs
 - So subset of Key-Value stores, where the value is searchable
 - Key = string/URL/path, unique for the entire data
 - Document = JSON, BSON, or XML – **structured** format
- Operations
 - **get/put** document by key
 - Retrieve document **by field** value

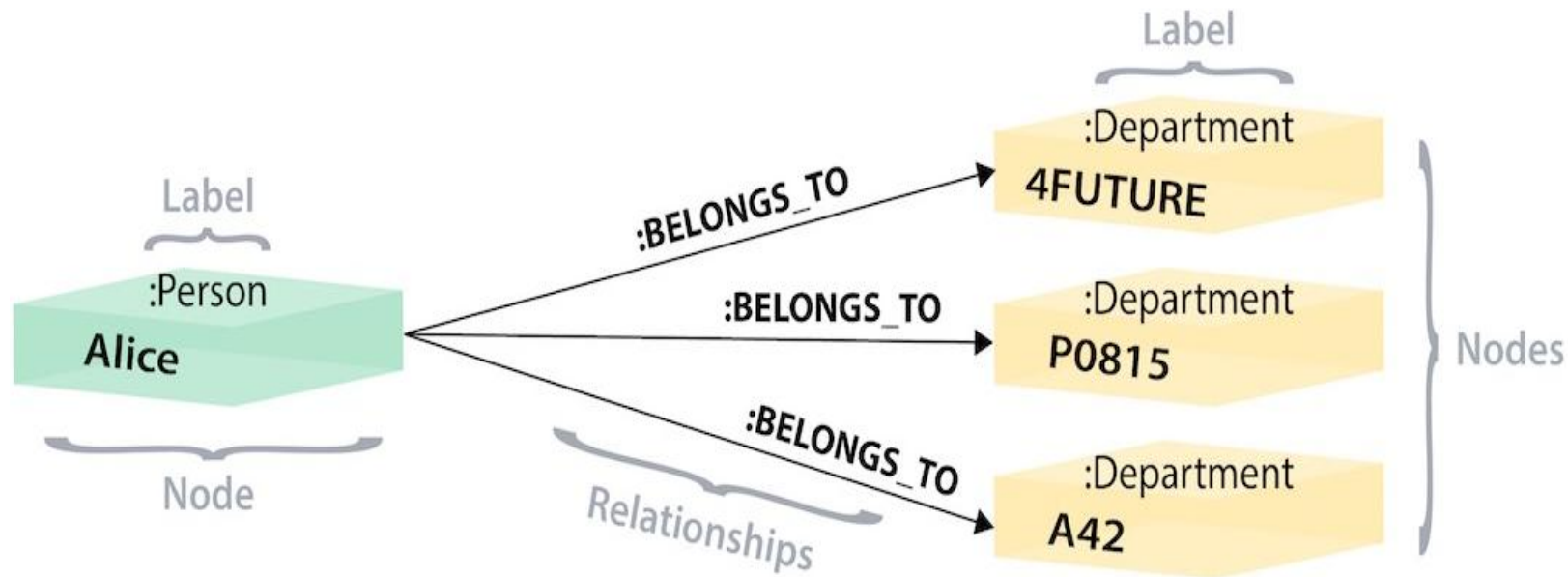
Graph Store

- Uses *graph structures* to store **data items** and **relationships** between them
- **Relationships** allow data to be linked together and retrieved with one operation
- Optimal for associative queries



Graph Store Features

- Relationships take first priority in graph databases
- They are stored directly and allow fast retrieval



Downsides of NoSQL

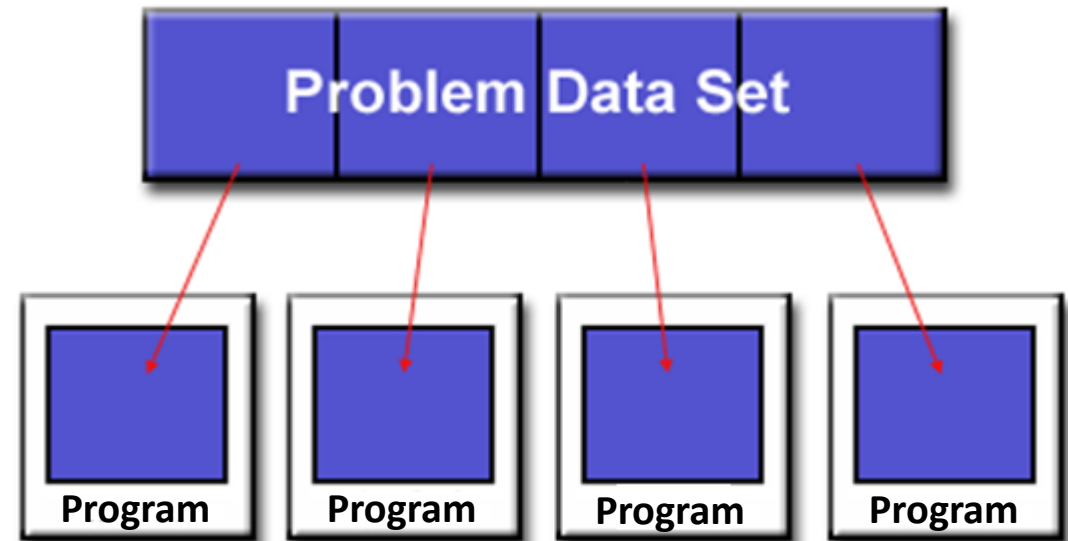
- There is **no universal query language** like SQL in NoSQL databases
 - But SQL is very powerful and expressive
- Each NoSQL product does things **differently!**
- **Relational databases are very mature** – lots of experience
 - 50+ years versus 10+ years
- **Relational databases are part of a vast “ecosystem”**
 - Many RDBMS tools available
 - Many RDBMS companies exist

NewSQL

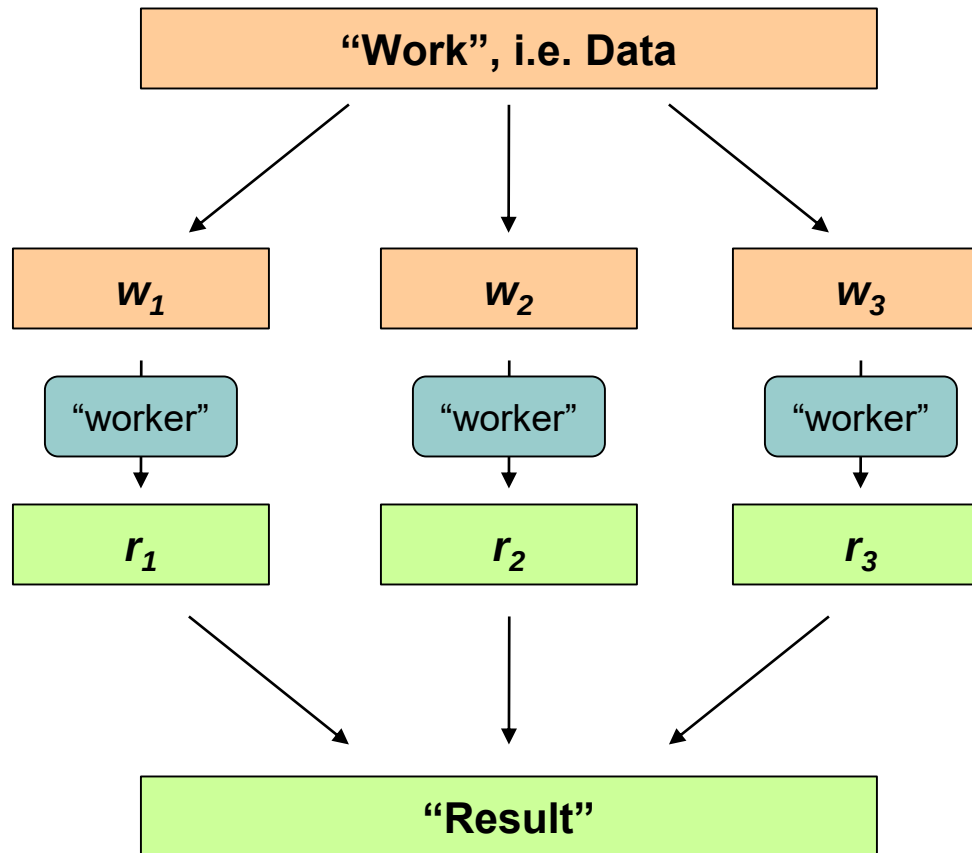
- **NewSQL** is a class of **modern RDBMS** that seeks to provide the same **scalable performance** of NoSQL systems for online transaction processing (read/write) workloads, while still maintaining the ACID properties of traditional database systems
- These systems break through conventional RDBMS performance limits
 - By employing **NoSQL-style features** such as **column-oriented** data storage and **distributed architectures**
 - By employing technologies such as **distributed computing**

Data Parallelism

- **Big Data** applications are based on the concept of **data parallelism** – splitting up data into smaller chunks, and processing these smaller chunks of data on **separate computers**
- Each computer executes the **same program** using just a portion of the data set
- **MapReduce** works like this



Divide and Conquer



Divide the work



**Give work to workers,
i.e. virtual machines**



Combine the results

Data Parallelism in the Cloud

- There are two core components of data parallelism:
 - **Distributed File System** – e.g. Hadoop Distributed File System
 - Flexible **analytical framework** – e.g. MapReduce
- Also a **massive infrastructure** is required to manage and control 100,000s of interconnected servers
- Google invented MapReduce
 - A **framework** which they used to scale their data processing needs

Data-intensive Applications in the Cloud

- Data-intensive information processing applications need **scalable** approaches to processing huge amounts of information (Tens of terabytes and even hexabytes of data)
- In cloud computing, the focus today is mostly on **MapReduce**, which is presently the most accessible and practical means of computing at this scale
- But other approaches are available as well

MapReduce Overview

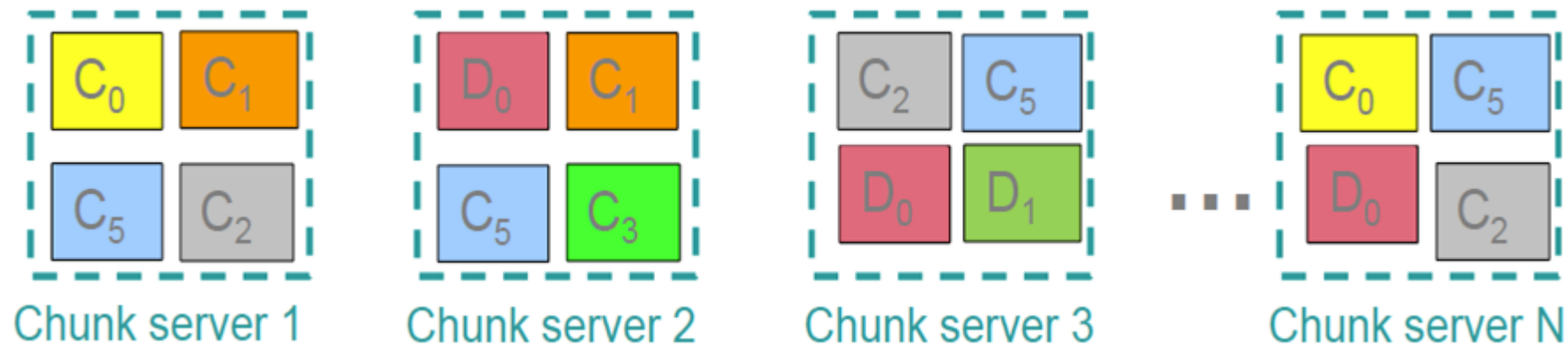
- **MapReduce** is a programming model for processing big data sets by using **data partitioning**, and
- A **parallel and distributed** implementation, capable of using an extremely large number of servers and hard drives
- Pioneered by Google (which processes > 20 PB of data per day)
- Popularized by the open-source project **Hadoop**
 - **Hadoop** is the implementation of Google's MapReduce framework by the Apache Software Foundation
 - Used by Facebook, Amazon, and many more ...

MapReduce Overview (continued)

- The power of **MapReduce** lies in its ability to scale to 100s or 1000s or 10,000s of servers, each with several processor cores
 - Web-scale data on the order of 100s of TBs or PBs
- It is highly likely that an application's input **data set will not fit** on a single server's hard drive
 - Hence, a **distributed file system** (e.g. the Hadoop Distributed File System – HDFS) is also required
 - The files are distributed over many hard disk drives

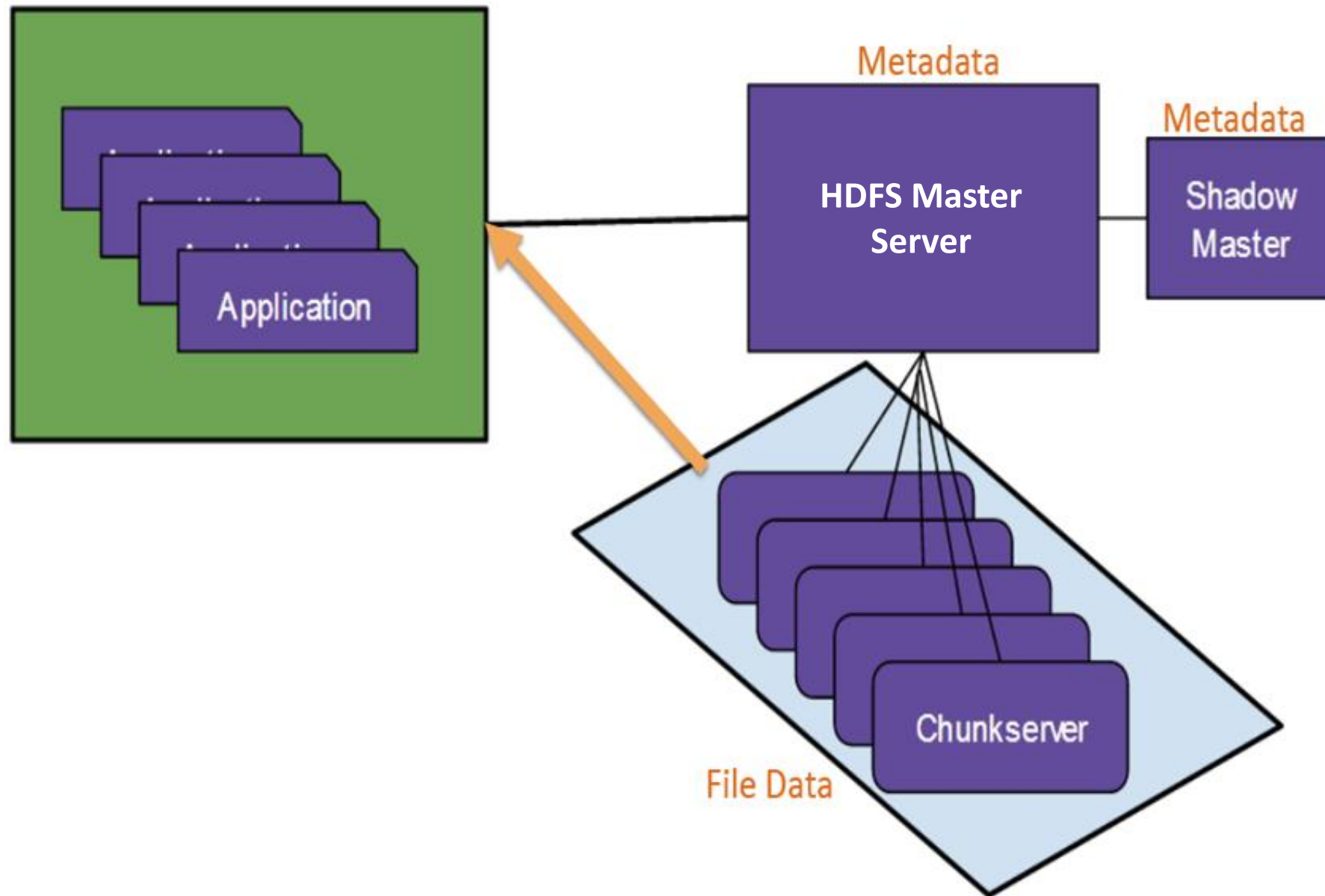
Distributed File System

- Reliable, **distributed file system** for petabyte-scale data
- Data kept in “**chunks**” spread **across** lots of **servers**
- Each chunk is **replicated** 3+ times on **different** chunk **servers**
 - To allow seamless **recovery** from disk or server failure



Distributed File System (continued)

- **Chunk Servers** – a big data file is split into contiguous “chunks”
 - Typically each chunk is 64 MB
 - Each chunk replicated (usually 3 or more times)
 - Try to keep replicas in different, **spatially-separate servers**
- **Master Server**
 - Stores **data** about location of the chunks
 - Is itself **replicated** to a “Shadow” master server
- **Customer programs** for file access
 - Communicate with master to find chunk servers
 - Connect directly to chunk servers to access the data



MapReduce

- **MapReduce** is a software **framework** for writing applications that process vast amounts of data (multi-terabyte data sets) **in parallel** on large clusters (thousands of servers) in a reliable, fault-tolerant manner
- MapReduce comprises two steps – the **Map** and **Reduce** steps
- MapReduce partitions the input data set into independent data chunks which are processed by a set of **identical Map programs** in a completely parallel manner
 - **Data parallelism** – same program operating on many different data chunks
 - For a given job, there may be 100s or 1000s of **Map programs**, running in parallel, on many servers

MapReduce (continued)

- The framework then sorts the outputs of the **Map** programs, which are then input to a set of **Reduce** programs
- The **Reduce** programs combine the individual results of the **Map** step to obtain the **final answer**
- An open-source MapReduce implementation is **Hadoop** – a highly-parallel, distributed framework, which required an associated development of a distributed file store, **HDFS**
 - Programs for **big data analysis**, data mining, etc
 - An “ecosystem” of cloud utility programs

MapReduce in Practice

- When we start a **Map/Reduce workflow**, the program will **split** the data input into segments, passing each segment to a different virtual machine (VM) on a different server
- Each VM then runs the **Map program** on that portion of data given to it
- The purpose of the **Map program** is to map the data into **<key, value> pairs** for the **Reduce program** to aggregate
 - So, the **Map** program (**which you write**) takes some input data, and maps it to **<key, value>** pairs according to your specifications

The Map Program

- **Map** programs require non-conventional way of thinking:
 - For example, assume you wish to find the word frequency in the collected works of your country's most famous author. You have the text in many, many files
 - What is the simplest code we could write? Remember that we may have the same program running on 100s of thousands of servers – we do not worry about optimal logic!
- Answer – can have **<word, count>** as our <key, value> pair
 - Our Map program, then, would output a **<word, 1>** pair **for each word** in the input stream

The Reduce Program

- Note that **Map programs** perform **no aggregation** (i.e. counting)
 - This is what the **Reduce program** is for
- After the Map programs finish, their outputs, i.e. **<key, value> pairs**, are then “**shuffled**”, and pairs with the **same key** (e.g. same word) are passed to a single instance of a **Reduce program**
 - There is more than one instance running the Reduce program
- The **Reduce program** (which you also write) takes a collection of **<key, value> pairs** and “reduces” them according to the code specified in the Reduce program

MapReduce Simple Example

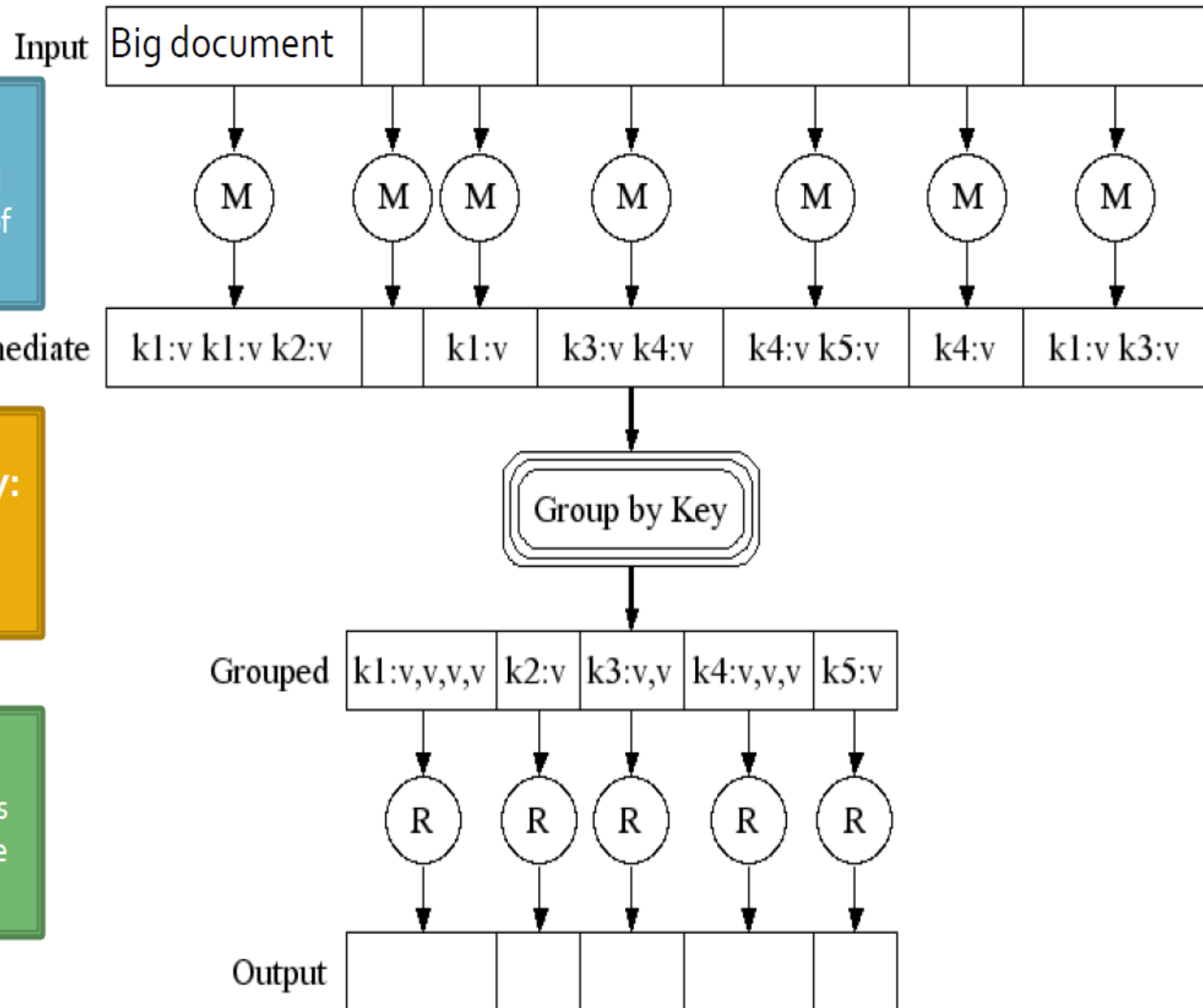
- In our word count example, we want to count the number of word occurrences so that we can get frequencies
 - Our **Map program**, then, would output a **<word, 1>** pair **for each word** in the input stream
 - Thus, we would want our **Reduce program** to simply sum the values of the collection of **<key, value>** pairs which have the same key, i.e. word

MAP:
reads input and
produces a set of
key value pairs

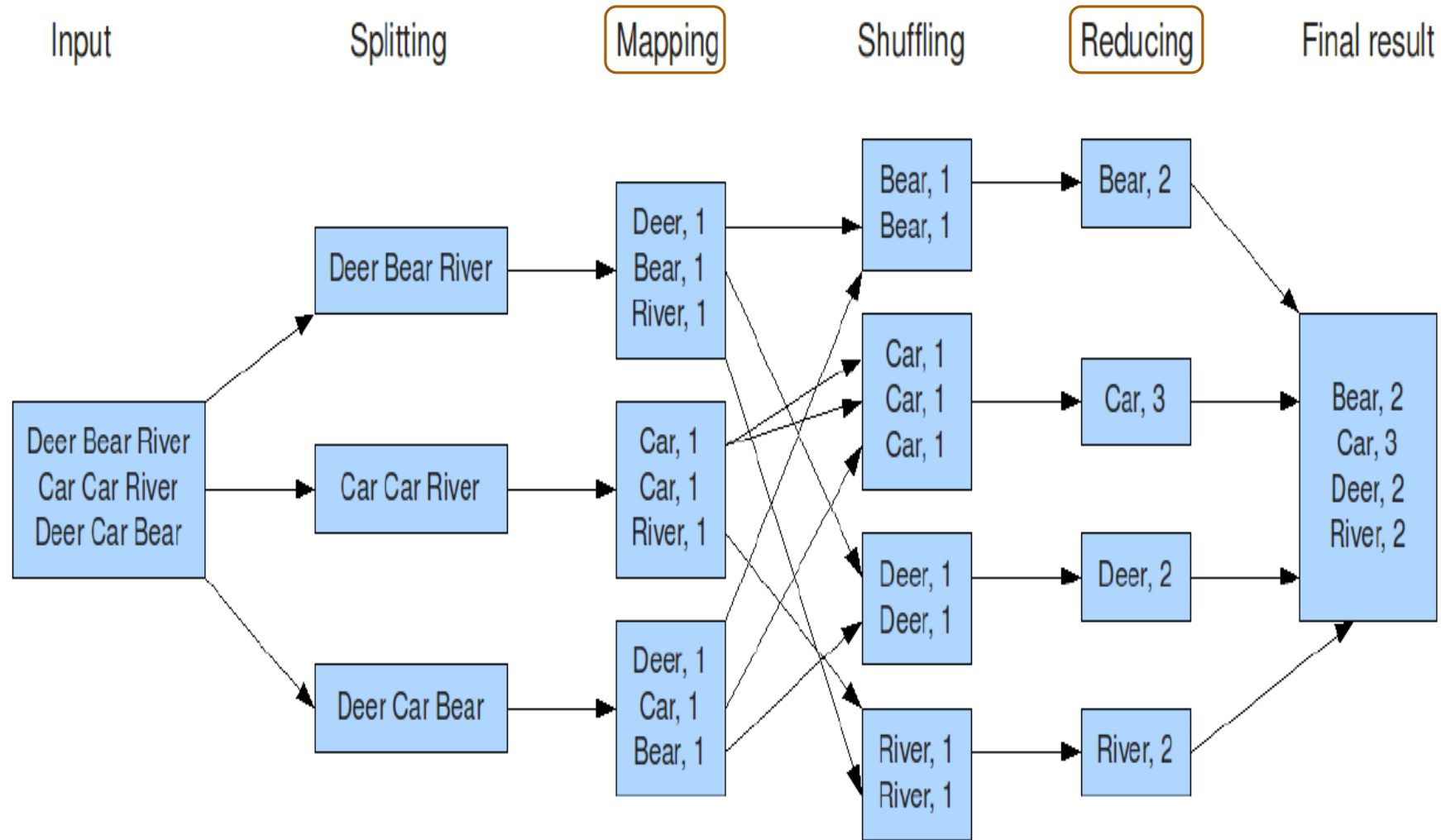
Intermediate

Group by key:
Collect all pairs
with same key

Reduce:
Collect all values
belonging to the
key and output



The overall MapReduce word count process



Questions?

Q & A